

SWT-CP: Lightweight Trap-Based Free-Rider Mitigation in Federated Learning

Aashnan Rahman

Department of Computer Science and Engineering

Islamic University of Technology

Student ID: 231041006

Abstract—Free riders consume a federated model while avoiding genuine local training, yet the server normally observes only their submitted parameters. This paper presents Suspicion-Weighted Trap Detection with Cumulative Penalties (SWT-CP), a lightweight server-side challenge–response framework. Selected clients secretly receive randomized trap models; returned updates are evaluated using a two-sided robust norm score and a scale-free layer profile. An initial two-trap coverage sweep is followed by candidate confirmation, frequent suspicion probing, rotating anchor panels, continued surveillance, cumulative penalties, and evidence-based rehabilitation. The detector uses no client-reported accuracy, loss, training time, raw data, auxiliary neural detector, or attack label.

We evaluate SWT-CP v8 on MNIST with 100 clients and 100 rounds, IID and Dirichlet non-IID partitions ($\alpha = 0.5$), attacker shares of 30% and 40%, seeds 42 and 43, and four attacks. All 560 free riders were removed across the IID matrix; the 40%, seed-43 configuration achieved 100% precision and recall. FR1–FR3 also retained 100% recall under non-IID data. Adaptive FR4, however, revealed a reproducible contamination boundary: it was fully detected at 30% non-IID attackers for both seeds, but 0/40 attackers were removed at 40% for either seed. Undetected attackers entered the anchor roster and could form malicious-majority reference panels. Final accuracy remained 98.82–98.95%, showing that model utility does not reveal detection failure. SWT-CP is therefore an effective lightweight detector within a measured operating region, but not a proof of client computation.

Index Terms—federated learning, free riding, anomaly detection, robust statistics, client auditing

I. INTRODUCTION

Federated learning (FL) trains a shared model without centralizing client data: a server broadcasts a global model, clients optimize locally, and returned parameters are aggregated by algorithms such as FedAvg [1]. This architecture supports large deployments [2], [3], but creates a verification asymmetry. The server sees an update, not the computation that produced it.

A free rider exploits this asymmetry by receiving the collaborative model while contributing little or no genuine training. Replay, calibrated noise, historical averaging, and geometry-matching attacks can appear superficially plausible [4], [5], [6]. The problem is harder under non-independent and identically distributed (non-IID) data because legitimate updates may be statistically unusual [7], [8].

Learned detectors and inference-based audits can obtain richer evidence but add training, access, or deployment assumptions. FRAD uses a DAGMM-style detector with contribution and reputation models [9]; PASS combines parameter auditing

and contribution evaluation [10]; FRIDA repurposes membership and property inference to test for evidence of training [11]. Hardware and cryptographic verification can provide stronger guarantees at substantial deployment cost. We instead ask whether an unpredictable server intervention and longitudinal statistical evidence can provide a useful lightweight operating point.

This paper makes three contributions:

- 1) a trap-based, attack-independent detector combining two-sided norm and layer-profile evidence with client states, penalties, and rehabilitation;
- 2) a rotating-anchor and continued-surveillance schedule that requires no additional client objective or self-reported metric; and
- 3) a 32-run IID/non-IID evaluation that reports both strong detection and a replicated FR4 failure caused by reference contamination.

II. THREAT MODEL AND ATTACKS

At round t , client i receives $w_{t,i}^{\text{send}}$ and returns $w_{t,i}^{\text{ret}}$. The server derives

$$\Delta_{t,i} = w_{t,i}^{\text{ret}} - w_{t,i}^{\text{send}}, \quad r_{t,i} = \|\Delta_{t,i}\|_2. \quad (1)$$

For a probe, $w_{t,i}^{\text{send}}$ is a randomized trap rather than the global model. Clients are never given a trap indicator. Trap-derived updates are excluded from aggregation.

FR1 replays the last model observed by that client. FR2 adds bounded uniform noise to the received model. FR3 returns an average of the five latest received models. FR4 predicts an update from five consecutive received-model differences and adds calibrated Gaussian noise. Traps enter attack history like ordinary received models. The server applies one detector to every client and does not read the configured attack type or ground-truth label. Collusion, on–off behavior, partial honest training, and defense-aware adaptation are outside the current experiment.

III. SWT-CP v8

A. Two-signal evidence

For reference norms R , SWT-CP computes a median m_r , MAD d_r , and

$$z_{t,i}^{\text{norm}} = \frac{r_{t,i} - m_r}{\max(d_r, 0.05)}. \quad (2)$$

The magnitude signal fires when $|z_{t,i}^{\text{norm}}| > 3$. For floating tensor k , a scale-free layer profile is

$$p_{t,i,k} = \frac{\|\Delta_{t,i,k}\|_2}{\sum_j \|\Delta_{t,i,j}\|_2}. \quad (3)$$

Its distance from a coordinate-wise median profile is standardized with MAD floor 0.01 and threshold 3. Either signal creates candidate evidence, but only simultaneous magnitude and profile failure earns nonzero magnitude penalty. A near-zero update ($r_{t,i} < 10^{-6}$) is separate strong evidence and adds five permanent points.

B. Coverage and surveillance

Rounds 1–10 are warm-up: only the near-zero check operates. Initial coverage constructs two independent traps and two randomized population passes. Each trap remains fixed for its pass, while two groups are probed per round; every client therefore receives both traps over ten rounds. Decisions are delayed until each pass is complete and calibrated separately, preventing different trap models from sharing one baseline.

Thereafter, every unflagged client belongs to one of ten surveillance groups. One group per round receives the current surveillance trap and is excluded from aggregation. An anomalous client becomes a candidate and permanently loses anchor eligibility. This track continues even when no suspects exist.

C. Candidate, suspect, and anchor processing

Clients occupy unflagged (U), candidate (C), or suspicious (S) states. Candidates are quarantined and receive fresh-trap confirmation. A normal confirmation clears C ; two joint failures confirm S . Suspicion groups are formed as $\max(1, \lfloor |S|/10 \rfloor)$ and receive distinct fresh traps. Each group uses up to three anchors that received the same trap.

Eligible anchors are active, unflagged clients with no lifetime anomaly. v8 rotates through this roster before reusing anchors where possible, while all off-duty anchors remain in surveillance. At most ten anchors are active in one round. Rotation reduces dependence on a fixed panel, but does not prove roster honesty.

D. Penalty, rehabilitation, and removal

Joint evidence adds two points in initial coverage, one in surveillance, and three in candidate/suspicion probing. The removal threshold is 15. Nonzero evidence requires at least ten episode probes before removal. After ten probes, a client whose joint-failure rate is at most 10% is rehabilitated: its current magnitude episode is cleared, while exact-send-back points remain. Rehabilitation takes precedence if it becomes eligible simultaneously with nonzero removal. Every current trap recipient and unresolved candidate is withheld from aggregation.

The detector never trusts local accuracy, loss, computation time, or resource reports. Cosine similarity is logged diagnostically but cannot affect a decision.

IV. EXPERIMENTAL SETUP

We use MNIST with a convolutional model, 100 fully participating clients, 100 rounds, three local epochs, batch size 32, SGD ($\eta = 0.01$, momentum 0.9), and attacker shares 30% and 40%. Data are either IID or allocated by a Dirichlet distribution with $\alpha = 0.5$. Every distribution/share/attack combination is executed with seeds 42 and 43, producing 32 completed v8 runs.

Removal defines a positive detection. Precision is $TP/(TP+FP)$, recall is $TP/(TP+FN)$, and $F1 = 2TP/(2TP+FP+FN)$. Candidates at round 100 are reported separately because they have no subsequent confirmation round. We additionally record final test accuracy, round time, process memory, state transitions, and trap assignments.

V. RESULTS

A. Aggregate performance

Table I aggregates FR1–FR4 within each configuration. All IID free riders were removed. Seed sensitivity appears mainly in false removals: IID 40%, seed 43 is perfect, whereas seed 42 removes six honest clients. Non-IID 30% retains complete recall but produces nine or ten false removals. Non-IID 40% falls to 75% recall because FR4 is entirely missed.

B. Attack-level non-IID behavior

Table II reports “removed free riders / removed honest clients.” FR1–FR3 achieve complete recall throughout; FR3 has the highest false-positive burden. FR4 succeeds at 30% but fails at 40% for both seeds.

At 30%, joint-evidence rates among trapped FR4 observations are 76.01% and 81.28%, and all attackers are removed around rounds 30–32. At 40%, rates collapse to 0% and 2.86%. Seed 42 never flags an FR4 client; seed 43 flags 18 at least once but confirms none. This is not inadequate coverage: each attacker is trapped repeatedly and all 80 remain active at round 100.

The calibration distribution changes with contamination. Initial-coverage median FR4 norm z-scores move from $-2.65/-3.10$ at 30% to $-1.55/-1.81$ at 40%. In seed 42, median trapped raw norm is 0.0365 for FR4 versus 0.4824 for honest clients, yet the broad non-IID mixture and contaminated references prevent joint evidence. The theoretical robustness of a scalar median does not guarantee multivariate operational separation below 50% contamination.

C. Anchor-contamination trace

Undetected FR4 clients qualify as anchors. In non-IID 40%, seed 42, honest client 85 receives 33 anchor assignments during its suspicious episode; ten are malicious. At rounds 66, 68, and 73, two of three anchors are FR4 clients. Those exact panels produce joint failures and add nine points; with six prior points, the honest client reaches 15 and is removed. Rotation therefore distributes reference use but cannot make an unverified roster trustworthy.

TABLE I
AGGREGATE V8 RESULTS ACROSS FR1–FR4.

Partition	Attackers/seed	TP	FP	FN	Precision	Recall	F1	Mean accuracy
IID	30%/42	120	8	0	93.75%	100%	96.77%	98.95%
IID	30%/43	120	1	0	99.17%	100%	99.59%	98.92%
IID	40%/42	160	6	0	96.39%	100%	98.16%	98.86%
IID	40%/43	160	0	0	100%	100%	100%	98.89%
Non-IID	30%/42	120	10	0	92.31%	100%	96.00%	98.88%
Non-IID	30%/43	120	9	0	93.02%	100%	96.39%	98.88%
Non-IID	40%/42	120	7	40	94.49%	75%	83.62%	98.84%
Non-IID	40%/43	120	8	40	93.75%	75%	83.33%	98.88%

TABLE II
NON-IID RESULT BY ATTACK (TP/FP).

Attack	30%		40%	
	s42	s43	s42	s43
FR1	30/3	30/1	40/1	40/3
FR2	30/2	30/3	40/1	40/2
FR3	30/3	30/3	40/4	40/3
FR4	30/2	30/2	0/1	0/0

D. False positives and utility

Non-IID false positives recur on the same clients across attacks: client 65 is removed six times and client 74 five times under seed 42; client 81 is removed seven times and client 29 five times under seed 43. This pattern implicates client-specific non-IID geometry rather than attack-specific logic.

Final accuracy remains 98.82–98.95%. The failed FR4 runs still reach 98.84% and 98.88%, because small fabricated updates mainly dilute learning while 60 honest clients continue training. Accuracy is therefore not a proxy for successful detection.

E. Execution cost

Across the eight aggregate configurations, mean round time ranges from 40.8 to 59.6 seconds and estimated four-attack batch time from 4.72 to 6.62 hours. Peak process memory ranges from 780 MB to 3.24 GB. Most batches execute on CPU; the IID 40% batches record CUDA allocation but remain serial across simulated clients. FR4 40% non-IID retains nearly all clients and is correspondingly more expensive per late round than attacks removed around round 30.

VI. DISCUSSION AND LIMITATIONS

The evaluation supports SWT-CP as a lightweight detector for replay, random-update, and historical-average attacks across tested partitions. It also detects trajectory-based FR4 under IID and at 30% non-IID contamination. However, two seeds are insufficient for narrow confidence intervals, and seed choice strongly affects false removals. Only MNIST, one non-IID concentration, full participation, and always-on independent attackers are tested. No external defense was rerun under identical conditions, so the study makes no comparative-superiority claim.

Most importantly, anchor eligibility is negative evidence: “never flagged” is not “verified honest.” A future design should require repeated positive challenge consistency before anchor admission, evaluate several independent panels before assigning penalty, and exploit paired within-client responses to traps A and B to reduce cross-client non-IID confounding. A third 40% seed and an intermediate 35% FR4 run would test the observed contamination transition. CIFAR-10, dropout, on-off and partial-training attacks, collusion, and defense-aware attackers remain necessary extensions.

SWT-CP also requires access to attributable updates. Keeping raw data local does not make the method compatible with ordinary secure aggregation [12]; a practical private deployment needs a secure statistic or auditable probe protocol.

VII. CONCLUSION

SWT-CP combines secret traps, robust two-signal evidence, longitudinal client states, rotating anchors, cumulative penalties, and rehabilitation without training an auxiliary detector or trusting client-reported metrics. It removes every tested free rider under IID data and every FR1–FR3 attacker under non-IID data while preserving MNIST accuracy. The replicated FR4 40% non-IID failure establishes a clear boundary: statistical robustness and anchor rotation do not prevent concentrated adaptive clients from shaping the reference population. The method is a strong lightweight baseline within its measured operating region, and the failure trace provides a concrete direction for verified-anchor and within-client differential designs.

REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [2] K. Bonawitz, H. Eichner, W. Grieskamp *et al.*, “Towards federated learning at scale: System design,” in *Proceedings of Machine Learning and Systems*, vol. 1, 2019, pp. 374–388.
- [3] A. Hard, K. Rao, R. Mathews *et al.*, “Federated learning for mobile keyboard prediction,” *arXiv preprint arXiv:1811.03604*, 2018.
- [4] Y. Fraboni, R. Vidal, and M. Lorenzi, “Free-rider attacks on model aggregation in federated learning,” in *Proc. AISTATS*, 2021, pp. 1846–1854.
- [5] J. Lin, M. Du, and J. Liu, “Free-riders in federated learning: Attacks and defenses,” *arXiv preprint arXiv:1911.12560*, 2019.
- [6] Z. Zhu, J. Shu, X. Zou, and X. Jia, “Advanced free-rider attacks in federated learning,” in *NeurIPS Workshop on New Frontiers in Federated Learning*, 2021.
- [7] Y. Zhao, M. Li, L. Lai *et al.*, “Federated learning with non-IID data,” *arXiv preprint arXiv:1806.00582*, 2018.

- [8] P. Kairouz, H. B. McMahan *et al.*, “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [9] B. Wang, H. Li, X. Liu, and Y. Guo, “FRAD: Free-rider attacks detection mechanism for federated learning in AIoT,” *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024.
- [10] J. Wang, X. Chang, J. Mišić, V. Mišić, and Y. Wang, “PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack,” *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2023.
- [11] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, “FRIDA: Free-rider detection using privacy attacks,” *arXiv preprint arXiv:2410.05020*, 2024.
- [12] K. Bonawitz, V. Ivanov, B. Kreuter *et al.*, “Practical secure aggregation for privacy-preserving machine learning,” in *Proc. ACM CCS*, 2017, pp. 1175–1191.